CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

PROJECT REPORT

ON

LOAN APPROVAL PREDICTION SYSTEM USING MACHINE LEARNING

A Web-Based Application Developed Using Python and Django

Submitted in partial fulfilment of the academic requirements for the course

BCA

Submitted By:

Divya

UID: **24BCA10551**

Submitted To

Mr. Ravi Raj

Assistant Professor

Chandigarh University

Academic Session: 2026–2027

Abstract

Loan approval is a critical process in the banking and financial sector. Institutions evaluate applicants using factors such as income, employment status, credit history, loan amount and loan term. Manual evaluation can be time-consuming and may lead to inconsistent decisions.

The Loan Approval Prediction System is a machine-learning-based web application developed using Python and Django. It uses a historical loan dataset of 614 records and 13 attributes. After data preprocessing, a classification model is trained and evaluated using accuracy, precision, recall, F1-score and a confusion matrix. The Django interface presents a dataset preview, a missing-value summary, interactive visual analytics, the evaluation results and an applicant form that returns a predicted loan status (approved or rejected).

The system is developed for educational purposes and does not replace the complete credit assessment process of a financial institution.

Keywords: *Machine Learning, Loan Approval, Classification, Python, Django, Data Preprocessing, Confusion Matrix.*

Chapter 1: Introduction

1.1 Background

Banks and financial institutions receive a large number of loan applications. Before granting a loan, they must judge whether the applicant is likely to repay it. This judgement depends on financial and personal details of the applicant, and a purely manual process requires considerable time and effort, especially when the number of applications is high.

1.2 Overview of Machine Learning

Machine learning is a branch of artificial intelligence in which computer systems learn patterns from data and use them to make predictions. In supervised learning, a model is trained on labelled examples. Here, the input features are applicant details (such as income, education and credit history) and the target variable is Loan_Status. Once trained, the model classifies new applications according to the patterns learned from historical records.

1.3 Purpose of the Project

The purpose of this project is to build a practical application that combines machine learning with web development. It demonstrates how data can be cleaned, explored and used to train a model, and how the trained model and its evaluation results can be presented to users through a Django web dashboard.

Chapter 2: Problem Statement and Objectives

2.1 Problem Statement

Manual loan assessment is slow and difficult to standardise when many applications must be reviewed. Moreover, the performance of a machine learning model is hard to interpret for users who do not work with programming tools. The problem addressed here is to develop a web-based system that predicts loan approval using a trained classification model and presents the data analysis and evaluation results in an easy-to-understand dashboard.

2.2 Objectives

1. To analyse a historical loan application dataset.
2. To handle missing values and encode categorical variables.
3. To explore the data through summaries and interactive visualisations.
4. To train a classification model for loan approval prediction.
5. To evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
6. To integrate the trained model with a Django web application.
7. To provide a form for predicting the loan status of a new applicant.

2.3 Scope

The project covers data preparation, exploratory analysis, model training, evaluation and web-based presentation. It is an academic prototype. It does not verify applicant identity, obtain credit-bureau records or make legally binding lending decisions.

2.4 Expected Outcomes

The expected outcome is a working Django application that shows the dataset analysis, the evaluation metrics and the confusion matrix, and predicts the loan status of a new applicant from the submitted form details.

Chapter 3: Literature Review and Existing System

Machine learning is widely used in finance for credit-risk assessment, fraud detection and loan default prediction. Classification algorithms such as Decision Trees, Logistic Regression and Random Forests learn the relationship between applicant features and past decisions. Data preprocessing and proper evaluation are essential, because a model with high overall accuracy may still perform poorly on one of the classes.

Parameter	Existing (manual) system	Proposed system
Assessment	Manual review of each application	Model-based prediction
Speed	Time-consuming for large volumes	Prediction generated instantly
Data insight	Needs separate analysis tools	Dashboard with preview, charts and metrics
Consistency	May vary between reviewers	Same model and rules for every record
Interface	Depends on institution workflow	Django web interface with prediction form

Table 3.1: Comparison of existing and proposed systems

Chapter 4: System Requirements and Technology Stack

4.1 Technology Stack

Technology	Purpose
Python	Main programming language for data processing and model development
Django	Web framework for URL routing, views, templates and request handling
Pandas	Reading the dataset, handling missing values and preparing features
NumPy	Numerical operations and array-based computation
Scikit-learn	Data splitting, classification model training and evaluation metrics
Matplotlib	Generating the confusion matrix image
Plotly	Interactive charts with hover and zoom on the dashboard
Joblib	Saving and loading the trained model
HTML / CSS	Structure and styling of the web pages

Table 4.1: Technologies used

Python, Pandas, NumPy and Scikit-learn form the data and machine learning pipeline, while Django connects this pipeline to the browser through views and templates. Plotly and Matplotlib are used to produce the visual outputs shown on the dashboard.

4.2 Hardware and Software Requirements

Component	Recommended configuration
Processor	Intel Core i3 or equivalent and above
RAM	4 GB minimum; 8 GB recommended
Storage	At least 1 GB free space for the project, environment and dataset
Software	Python 3, Django, required libraries, web browser, VS Code or any code editor

Table 4.2: Hardware and software requirements

4.3 Functional Requirements

- Display a preview of the loaded dataset and a missing-value summary.
- Show interactive visual analytics of the data.
- Display model evaluation metrics and the confusion matrix.
- Accept applicant details through a form and predict the loan status.

4.4 Non-Functional Requirements

- **Usability:** simple dashboard that is easy to navigate.
- **Reliability:** the same preprocessing is applied during training and prediction.
- **Performance:** predictions are returned within a short time.
- **Maintainability:** code is organised so that modules can be modified independently.

Chapter 5: System Design and Architecture

5.1 System Architecture

The system consists of a dataset, a preprocessing stage, a trained machine learning model and a Django web application. Data is cleaned and prepared, the model is trained and evaluated on test data, and the saved model and its results are made available to users through the Django dashboard.

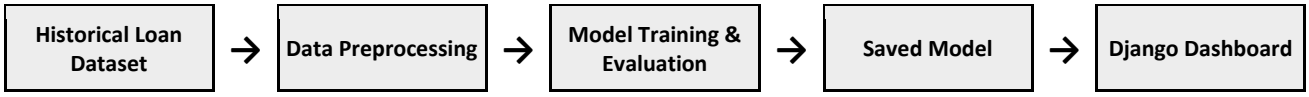


Figure 5.1: System workflow

5.2 Main Modules

Module	Description
Data handling	Loads the dataset and shows a preview of the records
Preprocessing	Handles missing values and encodes categorical features
Model training	Trains the classification model on the training split
Model evaluation	Calculates metrics and generates the confusion matrix
Visual analytics	Displays interactive charts for exploring the data
Prediction	Takes applicant details from a form and predicts the loan status

Table 5.1: System modules

5.3 Dataset Description

The dataset contains 614 historical loan application records with 12 input attributes and one target attribute (Loan_Status).

Attribute	Description
Loan_ID	Unique identifier of the loan application
Gender, Married	Gender and marital status of the applicant
Dependents	Number of dependents (0, 1, 2, 3+)
Education	Graduate or Not Graduate
Self_Employed	Whether the applicant is self-employed
ApplicantIncome	Income of the applicant
CoapplicantIncome	Income of the co-applicant
LoanAmount	Loan amount requested (in dataset units)
Loan_Amount_Term	Loan term in months
Credit_History	1 = good credit history, 0 = otherwise
Property_Area	Urban, Semiurban or Rural
Loan_Status (target)	Y = approved, N = rejected

Table 5.2: Dataset attributes

5.4 Data Flow

1. The historical dataset is loaded and previewed.
2. Missing values and data types are checked.
3. Missing values are filled and categorical columns are encoded.
4. Features and target are separated and split into training and testing sets.
5. The model is trained, evaluated and saved.
6. Django loads the saved model and displays the results and prediction form.

Chapter 6: Methodology and Implementation

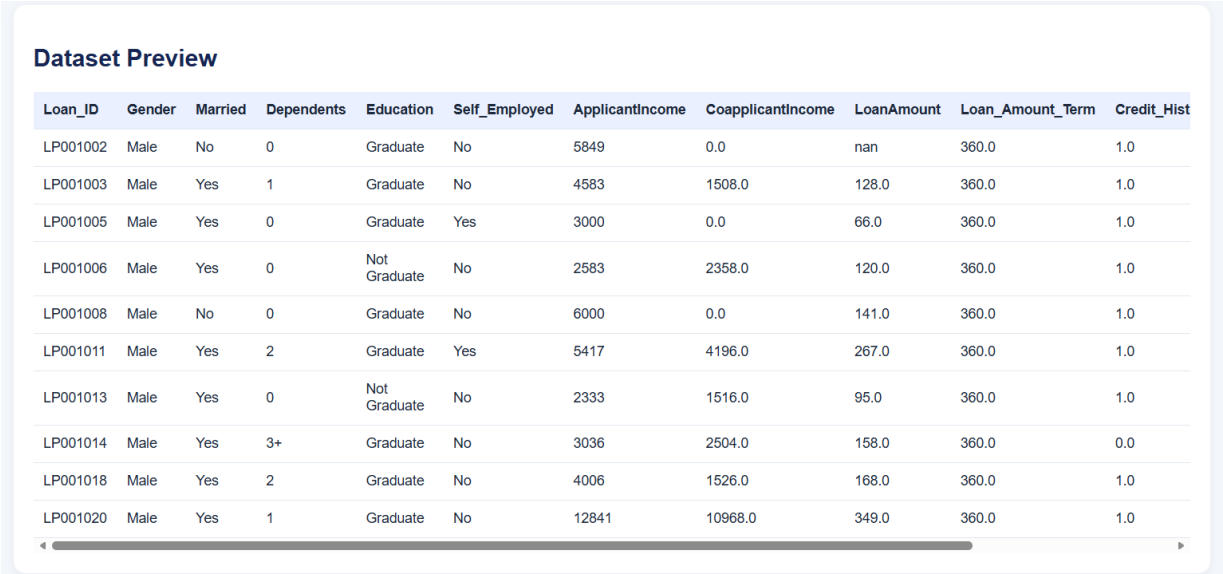
6.1 Project Methodology

The project follows these stages:

1. **Data collection:** obtain the historical loan dataset.
2. **Data exploration:** preview the records and check missing values.
3. **Preprocessing:** handle missing values and encode categorical variables.
4. **Data splitting:** separate features from the target and create training and testing sets.
5. **Model training and evaluation:** train the classifier and measure its performance.
6. **Django integration:** load the saved model and build the dashboard and prediction form.

6.2 Data Collection and Preview

The dashboard displays a preview of the dataset so that the structure and values of the records can be inspected.



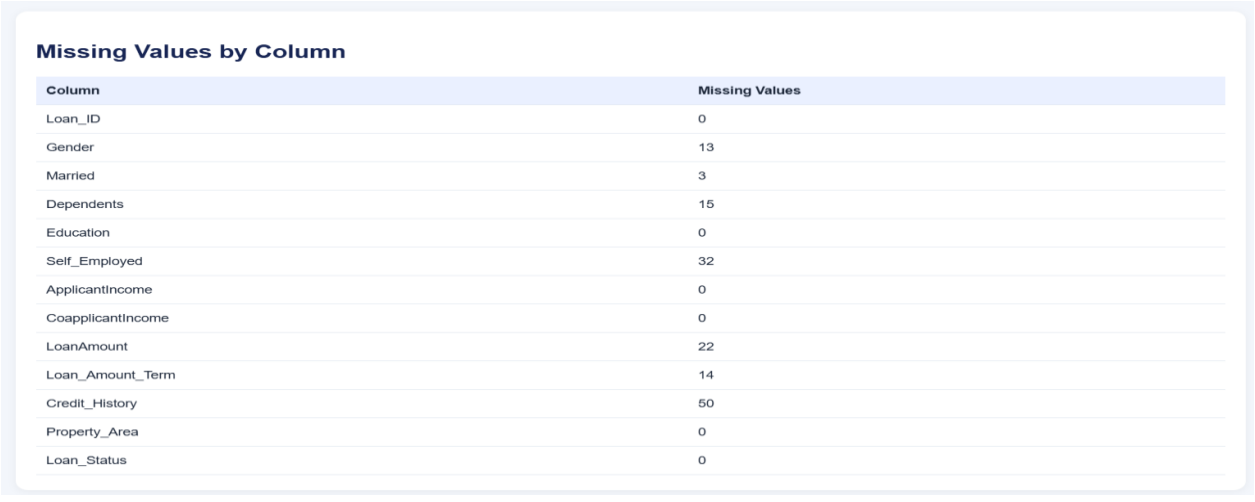
Dataset Preview

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_Hist
LP001002	Male	No	0	Graduate	No	5849	0.0	nan	360.0	1.0
LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0
LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0
LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0
LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0
LP001011	Male	Yes	2	Graduate	Yes	5417	4196.0	267.0	360.0	1.0
LP001013	Male	Yes	0	Not Graduate	No	2333	1516.0	95.0	360.0	1.0
LP001014	Male	Yes	3+	Graduate	No	3036	2504.0	158.0	360.0	0.0
LP001018	Male	Yes	2	Graduate	No	4006	1526.0	168.0	360.0	1.0
LP001020	Male	Yes	1	Graduate	No	12841	10968.0	349.0	360.0	1.0

Figure 6.1: Dataset preview on the dashboard

6.3 Handling Missing Values

The dashboard also summarises the number of missing values in each column. Seven columns contain missing entries: Gender (13), Married (3), Dependents (15), Self_Employed (32), LoanAmount (22), Loan_Amount_Term (14) and Credit_History (50), giving 149 missing entries in total. These are filled using suitable imputation before training.



Missing Values by Column

Column	Missing Values
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

Figure 6.2: Missing values by column

6.4 Exploratory Data Analysis

Interactive charts allow users to hover over the plots and zoom in to inspect values. The loan status chart shows that approved applications (Y) outnumber rejected ones (N) at roughly two to one, so the classes are imbalanced. This is why precision, recall and the confusion matrix are examined along with accuracy.

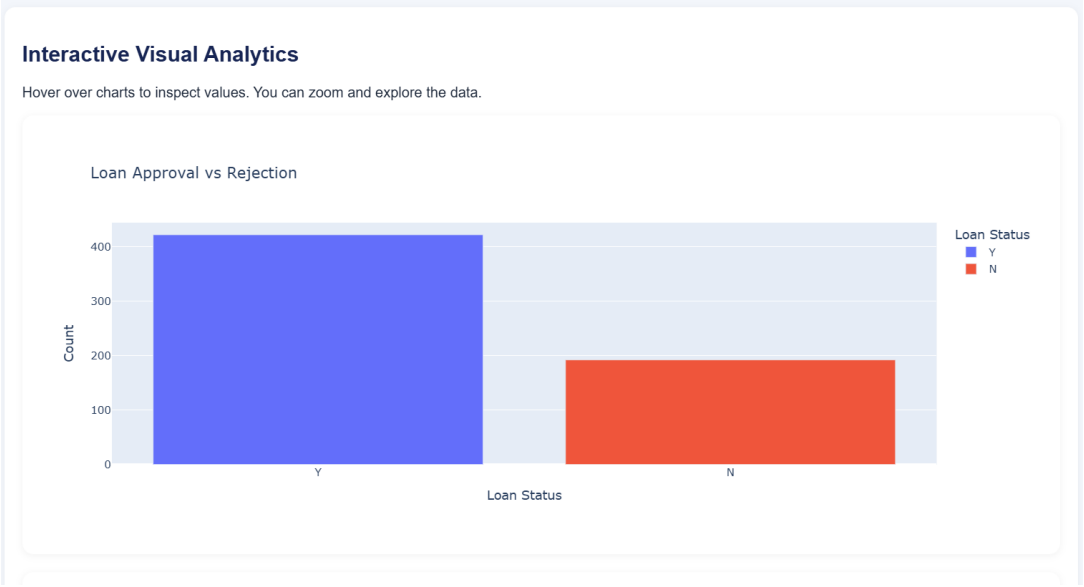


Figure 6.3: Loan approval vs rejection

6.5 Model Training

After preprocessing, the input features are separated from the target variable Loan_Status. The data is divided into training and testing subsets. The classification model is fitted on the training data and evaluated on the unseen test data. The trained model is then saved so that it can be reused without retraining.

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("loan_data.csv")
print(df.isnull().sum())            # missing values per column

X = df.drop(columns=["Loan_ID", "Loan_Status"])
y = df["Loan_Status"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Code Snippet 6.1: Data loading and train-test split

```
import joblib

joblib.dump(model, "loan_model.pkl")     # save the trained model
model = joblib.load("loan_model.pkl")    # load it inside the Django view
```

Code Snippet 6.2: Saving and loading the trained model

6.6 Django Implementation

Django is used to build the web application around the trained model:

- URL configuration maps browser requests to views.
- Views load the dataset and saved model, compute summaries and prepare data for display.
- Templates render the dataset preview, charts, metrics, confusion matrix and prediction form in HTML.
- The prediction view applies the same preprocessing used in training to the submitted form values before predicting.

Chapter 7: Results and Evaluation

7.1 Evaluation Metrics

The model is evaluated by comparing its predictions with the actual labels of the test data.

Metric	Formula	Meaning
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Proportion of all predictions that are correct
Precision	TP / (TP + FP)	Correct positive predictions among all positive predictions
Recall	TP / (TP + FN)	Actual positive cases correctly identified
F1-score	2 × P × R / (P + R)	Balance between precision and recall

Table 7.1: Evaluation metrics (TP, TN, FP, FN = true/false positives and negatives)

7.2 Model Results

Evaluation measure	Result
Dataset	Loan Prediction dataset (614 records)
Algorithm used	Apriori
Accuracy	0.862
Precision	0.840
Recall	0.988
F1-score	0.908

Table 7.2: Model evaluation results on the test set

7.3 Confusion Matrix

The confusion matrix compares actual and predicted classes and shows the number of correct and incorrect classifications for approved and rejected loans.

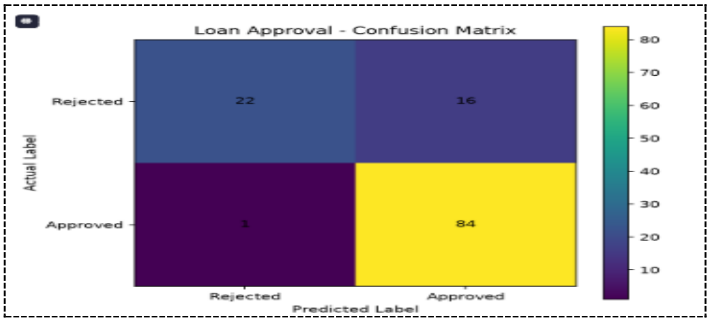


Figure 7.1: Confusion matrix displayed on the dashboard

7.4 Loan Status Prediction

The applicant details are entered in the form and, after clicking Predict Loan Status, the model returns whether the loan is likely to be approved or rejected.

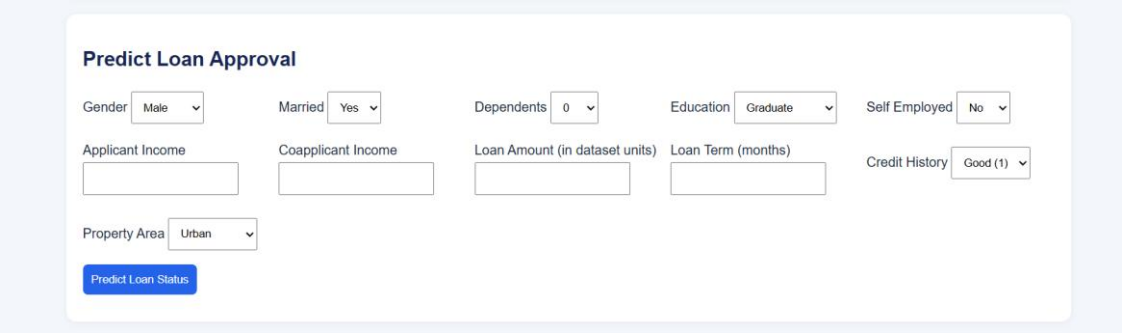


Figure 7.2: Loan approval prediction form

Chapter 8: Testing and User Manual

8.1 Test Cases

ID	Test scenario	Expected result	Status
TC01	Open the dashboard	Dashboard loads successfully	Pass
TC02	View dataset preview and missing values	Records and missing-value counts are displayed	Pass
TC03	Hover/zoom on interactive charts	Charts respond and show values	Pass
TC04	View metrics and confusion matrix	Metrics and matrix are visible	Pass
TC05	Submit valid applicant details	Predicted loan status is displayed	Pass
TC06	Submit the form with empty income fields	Error message / form validation shown	Pass
TC07	Change credit history and re-submit	Prediction is generated for the new input	Pass

Table 8.1: Functional test cases

8.2 User Manual

1. Open Command Prompt in the project directory and activate the virtual environment.
2. Start the server with the command: `python manage.py runserver`
3. Open `http://127.0.0.1:8000/` in a web browser.
4. View the dataset preview and the missing-values table.
5. Explore the interactive charts and review the evaluation metrics and confusion matrix.
6. In the Predict Loan Approval form, select the applicant details and enter the income, loan amount and loan term.
7. Click Predict Loan Status to view the predicted result.
8. Press `Ctrl + C` in the terminal to stop the server.

8.3 Limitations

- The model depends on the quality and size of the historical dataset and may not generalise to all applicants.
- Class imbalance (more approved than rejected records) can affect the predictions.
- The system uses only the dataset attributes and does not check documents or credit-bureau records.
- It is an academic prototype and does not replace human review or institutional lending policies.

8.4 Error Handling

- Empty or invalid form fields should show a clear message instead of a prediction.
- If the saved model file is missing, the application should report the problem to the user.
- Debugging should be turned off and a secure secret key used before any public deployment.

Chapter 9: Conclusion and Future Scope

9.1 Conclusion

The Loan Approval Prediction System demonstrates how machine learning can be integrated with a web application for loan classification. The project covers the complete workflow: loading and exploring a historical dataset, handling missing values, training and evaluating a classification model, and presenting the results through a Django dashboard with interactive charts and a prediction form.

The project gave practical experience in Python programming, data preprocessing, classification, model evaluation and web development. It also showed the importance of interpreting metrics such as precision, recall and the confusion matrix, instead of relying on accuracy alone, especially when the classes are imbalanced. The system should be treated as an educational decision-support prototype and not as a replacement for professional credit assessment.

9.2 Future Scope

1. Compare multiple classification algorithms on the same dataset.
2. Add ROC curves, AUC and cross-validation for stronger evaluation.
3. Add explainable AI features to show which inputs influenced a prediction.
4. Store prediction history in a database.
5. Add dataset upload and stronger input validation.
6. Assess the fairness of the model across different applicant groups.
7. Deploy the application securely on a cloud platform.

References

1. Django Software Foundation. Django Documentation. https://docs.djangoproject.com/
2. Python Software Foundation. Python Documentation. https://docs.python.org/3/
3. Scikit-learn Developers. Scikit-learn: Machine Learning in Python. https://scikit-learn.org/stable/
4. The pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/
5. NumPy Developers. NumPy Documentation. https://numpy.org/doc/
6. Plotly Technologies. Plotly Python Graphing Library. https://plotly.com/python/